

Python & FastAPI

Interview Questions & Answers

Senior / AI Engineer Level | 8+ Years Experience

Complete Edition — includes Fundamentals, Cross-Language Concepts, and Coding Exercises

1. Python — Beginner
2. Python — Intermediate
3. Python — Advanced
4. Python — Functions, Variables & Built-ins (from practice files)
5. Python — List Comprehension & Iterators
6. Python — OOP Fundamentals
7. Memory Management & Garbage Collection
8. Cross-Language Concepts (from Ruby; same in Python)
9. FastAPI — Beginner
10. FastAPI — Intermediate
11. FastAPI — Advanced
12. Senior Scenario Questions
13. Coding Questions
14. What Senior Interviewers Expect

1. Python — Beginner

Q: Q1. What is Python?

Python is a high-level, interpreted, general-purpose programming language known for readability, a rich ecosystem, and strong support for backend development, automation, data engineering, AI/ML, and scripting.

Why companies use it: simple syntax, fast development speed, huge ecosystem, strong backend and AI support, good for APIs, automation, and data work.

Q: Q2. Difference between list, tuple, set, and dictionary?

- **List** — ordered, mutable, allows duplicates: [1, 2, 3]
- **Tuple** — ordered, immutable, allows duplicates: (1, 2, 3)
- **Set** — unordered, mutable, unique values only: {1, 2, 3}
- **Dictionary** — key-value mapping, mutable, keys unique: {"name": "Hasan"}

Q: Q3. Mutable vs Immutable objects?

- **Mutable** (can be changed): list, dict, set
- **Immutable** (cannot be changed): str, tuple, int, float, bool

```
a = [1, 2] a.append(3) # allowed s = 'hello' # s[0] = 'H' # TypeError – string is immutable
```

Q: Q4. Difference between == and is?

- **==** checks value equality
- **is** checks object identity (same memory address)

```
a = [1, 2] b = [1, 2] print(a == b) # True print(a is b) # False
```

Q: Q5. What are *args and **kwargs?

- ***args** — collects extra positional arguments as a tuple
- ****kwargs** — collects extra keyword arguments as a dictionary

```
def demo(*args, **kwargs): print(args) # (1, 2) print(kwargs) # {'name': 'Hasan', 'city': 'Dubai'} demo(1, 2, name='Hasan', city='Dubai')
```

Q: Q6. Difference between / and //?

- **/** — normal division → returns float: 10 / 3 = 3.333
- **//** — floor division → returns int: 10 // 3 = 3

Q: Q7. What is truthy and falsy?

Falsy values: False, None, 0, 0.0, "", [], {}, set() — everything else is truthy.

Q: Q8. Difference between append() and extend()?

- **append(x)** — adds one item

- **extend(iterable)** — adds multiple items from iterable

```
a = [1, 2] a.append([3, 4]) # [1, 2, [3, 4]] b = [1, 2] b.extend([3, 4]) # [1, 2, 3, 4]
```

2. Python — Intermediate

Q: Q9. What is the LEGB rule?

Python resolves variable names in order: Local → Enclosing → Global → Built-in

```
x = 'global' def outer(): x = 'outer' def inner(): x = 'inner' print(x) # 'inner' inner()
outer()
```

Q: Q10. Why are mutable default arguments dangerous?

```
# BAD – default list created once and reused across calls def add_item(item, items=[]):
items.append(item) return items # GOOD def add_item(item, items=None): if items is None:
items = [] items.append(item) return items
```

Q: Q11. What is a decorator?

A function that wraps another function to add behavior without modifying the original code.

```
def log_decorator(func): def wrapper(*args, **kwargs): print('Before') result =
func(*args, **kwargs) print('After') return result return wrapper @log_decorator def
greet(): print('Hello')
```

Q: Q12. What is a closure?

A function that remembers variables from its outer scope even after the outer function has finished.

```
def outer(msg): def inner(): print(msg) return inner fn = outer('hello') fn() # prints
'hello'
```

Q: Q13. @staticmethod vs @classmethod?

- **@staticmethod** — no self/cls, behaves like a regular function inside class namespace
- **@classmethod** — receives class as first argument (cls), useful for factory/alternative constructors

```
class Person: def __init__(self, name): self.name = name @classmethod def from_dict(cls,
data): return cls(data['name'])
```

Q: Q14. Shallow copy vs deep copy?

- **Shallow copy** — copies outer object; nested objects still shared
- **Deep copy** — copies everything recursively (no shared references)

```
import copy a = [[1, 2], [3, 4]] b = copy.copy(a) # shallow c = copy.deepcopy(a) # deep
```

Q: Q15. What is a generator?

A function using yield to return values one at a time — memory efficient for large data.

```
def count_up_to(n): i = 1 while i <= n: yield i i += 1 for x in count_up_to(3): print(x) #
1, 2, 3
```

Q: Q16. return vs yield?

- **return** — ends function, returns final value
- **yield** — pauses function, returns value, preserves state for next call

Q: Q17. `sorted()` vs `list.sort()`?

- **`sorted()`** — returns a new sorted list, works on any iterable
- **`list.sort()`** — sorts in-place, returns None

Q: Q18. How does exception handling work?

```
try: x = 10 / 0 except ZeroDivisionError: print('Cannot divide by zero') else: print('No exception') finally: print('Always runs')
```

3. Python — Advanced

Q: Q19. What is the GIL?

GIL = Global Interpreter Lock. In CPython, only one thread executes Python bytecode at a time. It limits CPU-bound multithreaded performance but does not prevent concurrency for I/O-bound tasks.

Strong answer: "The GIL is a mutex in CPython that allows only one thread to run Python bytecode at a time. It limits CPU-bound multithreading, but for I/O-bound tasks Python achieves concurrency through threads or asyncio."

Q: Q20. Threading vs Multiprocessing vs Asyncio?

- **Threading** — I/O-bound synchronous tasks (API calls, DB, file I/O)
- **Multiprocessing** — CPU-bound tasks (image processing, heavy computation, ML preprocessing)
- **Asyncio** — High-concurrency I/O-heavy apps (web servers, many HTTP calls, streaming)

Q: Q21. What is an event loop?

Core engine behind asyncio. Schedules coroutines, handles waiting tasks, and resumes them when I/O completes. Task hits await → control returns to event loop → loop runs other tasks → original task resumes.

Q: Q22. What happens if you call blocking code inside async code?

It blocks the event loop, killing concurrency for all other requests.

```
# BAD async def handler(): data = slow_blocking_db_call() # blocks entire event loop
return data # GOOD - offload to thread pool: # await
asyncio.to_thread(slow_blocking_db_call)
```

Fixes: use async library, or use asyncio.to_thread() / run_in_executor() to offload to thread pool.

Q: Q23-24. What is a context manager? How to create a custom one?

```
class MyContext: def __enter__(self): print('Enter') return self def __exit__(self,
exc_type, exc_val, exc_tb): print('Exit') with MyContext(): print('Inside')
```

Q: Q25. What are dataclasses?

Reduce boilerplate for data-holding classes. Auto-generates __init__, __repr__, __eq__.

```
from dataclasses import dataclass @dataclass class User: id: int name: str
```

Q: Q26. What is MRO (Method Resolution Order)?

Order in which Python looks for methods in inheritance hierarchies (especially multiple inheritance). Inspect via ClassName.mro().

Q: Q27. Good Python backend project structure?

```
app/ api/routes/ services/ repositories/ models/ schemas/ core/ config.py logging.py db/
tests/
```

routes → HTTP only | services → business logic | repositories → DB | schemas → request/response contracts

4. Python — Functions, Variables & Built-ins

This section captures core concepts directly from practice exercises — great refreshers for interviews.

Q: Q28. How does Python variable assignment work?

```
x = 10 # single assignment print(x) # 10 a, b, c, d = 10, 20, 30, 40 # multiple assignment
print(b) # 20 print(d) # 40
```

Q: Q29. What are Python's core data types?

```
a = True type(a) # bool b = 99 type(b) # int type(0.99) # float print(float(b)) # 99.0
print(int(0.99)) # 0
```

Q: Q30. How do strings and type conversion work?

```
m = 100 print(str(m) + ' days') # '100 days' print("It's cool, isn't it?") # single vs
double quotes print('Click "OK"')
```

Q: Q31. Arithmetic operators — what to know?

```
15 + 23 # addition 26 - 50 # subtraction 20 / 4 # division → float: 5.0 22 / 4 # 5.5 22
// 4 # floor division → 5 2 ** 3 # exponentiation → 8 # String indexing "Friday"[4] # 'd'
"Bingo"[0] # 'B'
```

Q: Q32. What are comparison operators?

```
10 == 10 # True 10 != 1 # True 100 > 5 # True 200 < 3 # False 10 >= 2 # True 1 <= 20 #
True 5 is 10 # False (identity) 5 is not 6 # True
```

Q: Q33. How do logical operators work?

```
True and True # True True and False # False False or True # True not True # False 3 > 5
and 4 > 10 # False True and not True # False False or not True and True # False
(precedence: not > and > or)
```

Q: Q34. How do conditional statements work in Python?

```
x = 2 if x > 200: print("Big") elif x > 100 and x <= 200: print("Average") elif x > 0 and
x <= 100: print("Small") elif x < 0: print("Negative")
```

Q: Q35. What is the difference between a function and a method?

A key distinction that interviewers test:

- **Function** — defined outside a class, called directly: `greet()`
- **Method** — defined inside a class, called using object: `obj.greet()`
- Functions have no implicit `self` parameter; methods have `self` (for instance methods)
- Functions are independent; methods are associated with an object

Q: Q36. What are Python's core built-in functions?

```
type(10) # int int(5.0) # 5 float(3) # 3.0 str(500) # '500' max(10, 20) # 20 min(1, 2) # 1
abs(-1) # 1 sum([1,2,3,4]) # 10 round(3.2) # 3 pow(2, 10) # 1024 len("Hasan") # 5
```

Q: Q37. How does list manipulation work in Python?

```
numbers = [10, 25, 40, 50] numbers[-3] # 25 (negative indexing) numbers[0] = 15 # update
del numbers[1] # delete numbers.append(60) # push to end: [10, 25, 40, 50, 60]
numbers.extend([70,80]) # add multiple: [..., 70, 80] len(numbers) # length numbers[1:3]
# slice [25, 40] numbers[:2] # first 2 items numbers[-2:] # last 2 items numbers.index(50)
# 2 (find index) [2,1,3].sort() # in-place sort [2,1,3].sort(reverse=True) # descending
```

Q: Q38. What is a tuple and when should you use it?

```
cars = ("BMW", "Dodge", "Ford") cars[1] # "Dodge" # Tuples are immutable – cannot update
by index like lists # Use case: return multiple values from a function def
rectangle_info(length, width): area = length * width perimeter = 2 * (length + width)
return area, perimeter result = rectangle_info(2, 10) print(result[0]) # area: 20
print(result[1]) # perimeter: 24
```

Q: Q39. How do dictionaries work in Python?

```
menu = {'meal_1': 'Spaghetti', 'meal_2': 'Fries'} print(menu['meal_2']) # Fries
menu['meal_5'] = 'Soup' # add new key menu['meal_2'] = 'Burger' # update existing
price_list = {'Burger': 8, 'Fries': 5} print(price_list.get('Fries')) # 5 (safe access –
returns None if missing)
```

5. Python — List Comprehension & Iterators

Q: Q40. What is list comprehension and when should you use it?

List comprehension is a concise, Pythonic way to create lists from iterables. It replaces verbose for loops with a single expression.

```
numbers = [1,2,3,4,5,6,7,8,9] # Non-Pythonic: new_numbers = [] for n in numbers:
new_numbers.append(n * 2) # Pythonic: new_numbers = [n * 2 for n in numbers] # [2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Q: Q41. How do nested list comprehensions work?

```
# Nested loops in one expression: computed = [i + j for i in range(2) for j in range(5)] #
[0, 1, 2, 3, 4, 1, 2, 3, 4, 5] # Multi-dimensional (produces nested lists): nested = [[i+j
for i in range(2)] for j in range(5)] # [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5]]
```

Q: Q42. List comprehension with conditional logic?

```
# Filter: only even numbers * 10 from 1-10 print([n * 10 for n in range(1, 11) if n % 2 ==
0]) # [20, 40, 60, 80, 100] # Ternary: multiply if even, 'None' if odd print([num * 10 if
num % 2 == 0 else 'None' for num in range(1, 11)]) # ['None', 20, 'None', 40, 'None', 60,
'None', 80, 'None', 100]
```

Q: Q43. How do for loops and iterators work in Python?

```
digits = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] for n in digits: print(n) # Print on same line
using end parameter: for d in digits: print(d, end="") # 0123456789
```

Q: Q44. How do while loops work and when do you use range()?

```
# Print even numbers x = 0 while x <= 20: print(x) x += 2 # Using range:
print(list(range(1, 11))) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] for n in range(20): if n % 2
== 0: print(n) # prints even numbers
```

Q: Q45. How do you iterate over a dictionary?

```
prices = {"spaghetti": 4, "lasagna": 5, "hamburger": 2} quantity = {"spaghetti": 6,
"lasagna": 10, "hamburger": 0} money_spent = 0 for item in prices: if prices[item] >= 5:
money_spent += quantity[item] * prices[item] print(money_spent) # 50
```

6. Python — OOP Fundamentals

Q: Q46. What is OOP and what are the four pillars?

- **Encapsulation** — bundling data and methods inside a class, hiding internal state
- **Inheritance** — a class deriving properties from a parent class
- **Polymorphism** — the same interface (method name) behaving differently based on object type
- **Abstraction** — exposing only essential features, hiding implementation details

Q: Q47. How do instance variables vs class variables differ?

```
class Counter: count = 0 # class variable – shared across all instances
def __init__(self, name):
    self.name = name # instance variable – unique to each instance
Counter.count += 1
a = Counter('Alice')
b = Counter('Bob')
print(Counter.count) # 2
print(a.name) # 'Alice'
```

Q: Q48. What is method overriding?

A subclass providing its own implementation of a method defined in a parent class. The subclass method takes precedence at runtime.

```
class Animal:
    def speak(self):
        return 'Some sound'
class Dog(Animal):
    def speak(self):
        # overrides parent
        return 'Woof!'
d = Dog()
print(d.speak()) # 'Woof!'
```

Q: Q49. What is encapsulation and how do you enforce it?

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # private (name-mangled)
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
    def get_balance(self):
        return self.__balance
```

Note: Single underscore `_x` is a convention for 'protected'. Double underscore `__x` is name-mangled to `__ClassName__x`.

Q: Q50. What is the difference between global and instance variables?

```
# Global variable – accessible everywhere in the module
global_count = 0
class MyClass:
    def set_val(self):
        global global_count
        global_count = 10
# Instance variable – unique to each object instance
class Practice:
    def some_code(self):
        self.name = 'Hasan' # stored on instance
    def other_code(self):
        print(self.name) # accessible across methods
```

7. Memory Management & Garbage Collection

Q: Q63. How does Python manage memory?

Python uses automatic memory management via reference counting and a cyclic garbage collector. You do not manually free memory — Python handles this for you.

- **Reference counting** — every object has a count of references pointing to it. When count hits 0, memory is freed immediately.
- **Cyclic GC** — handles circular references that reference counting cannot free.

```
import gc
gc.collect() # manually trigger garbage collection
```

Q: Q64. Stack vs Heap memory — what is the difference?

- **Stack** — very fast, local to function calls, freed automatically when function ends, very limited in size
- **Heap** — slightly slower, no automatic cleanup (GC handles it), size limited only by available RAM

Python objects are always allocated on the heap. If an object is smaller than 40 bytes, its content can be embedded directly; otherwise it points to a separate memory segment.

Q: Q65. What is the Mark-and-Sweep algorithm (CPython)?

- **Mark phase** — scans all existing objects, marks those that can be cleaned up. Halts code execution.
- **Sweep phase** — runs in the background, frees marked objects. Memory available after sweep.

Note: The mark phase pause is where production issues can occur. This is why minimizing object creation in hot paths matters.

Q: Q66. How does Python's memory differ from C?

In C, you manually allocate (malloc) and free (free) memory. In Python, memory is managed automatically. Python provides 'infinite' memory from the developer's perspective — you never call free manually. This is the tradeoff: developer productivity vs. fine-grained control.

8. Cross-Language Concepts (Fundamentals Are Universal)

These concepts were documented from Ruby experience but apply equally to Python. Understanding the 'why' behind these patterns makes you a stronger engineer in any language.

Q: Q51. map vs each/for-each — what is the key difference?

- **map** — transforms each element, returns a new collection with the modified values
- **for/each** — iterates to produce side effects; returns the original collection unchanged

```
# Python equivalents: nums = [1, 2, 3, 4] # map: returns modified values result =  
list(map(lambda x: x * 2, nums)) # [2, 4, 6, 8] # or using list comprehension (more  
Pythonic): result = [x * 2 for x in nums] # for: side effects only, original unchanged for  
x in nums: print(x * 2) # prints 2 4 6 8 – but nums is still [1,2,3,4]
```

Q: Q52. select/filter — how does it work?

- **filter/select** — returns only elements where the condition is True

```
nums = [1, 2, 3, 4, 5, 6] # filter built-in: evens = list(filter(lambda x: x % 2 == 0,  
nums)) # [2, 4, 6] # list comprehension (Pythonic): evens = [x for x in nums if x % 2 ==  
0]
```

Q: Q53. Truthy and falsy — how do they differ between Python and Ruby?

This is a key difference: in Python, 0 and [] are falsy. In Ruby, only nil and false are falsy — 0 and [] are truthy.

```
# Python: these are ALL falsy: if not 0: print('falsy') # falsy if not []: print('falsy')  
# falsy if not "": print('falsy') # falsy if not None: print('falsy') # falsy # Ruby  
(reference for comparison): 0 is TRUTHY in Ruby # if 0 puts 'TRUE' => prints TRUE in Ruby
```

Q: Q54. Splat / unpacking operator — what is it?

Python uses * to unpack iterables and ** to unpack dictionaries — similar to Ruby's splat operator.

```
# Collect variable positional args: def accept_as_array(*players): print(players)  
accept_as_array('ronaldo', 'messi') # ('ronaldo', 'messi') # Unpack in function calls:  
def add(a, b, c): return a + b + c nums = [1, 2, 3] print(add(*nums)) # 6 # Dict  
unpacking: data = {'name': 'Hasan', 'age': 28} def show(name, age): print(name, age)  
show(**data) # Hasan 28
```

Q: Q55. Strongly typed vs weakly typed languages?

Python is strongly typed — it does not silently coerce types. Ruby is weakly typed — implicit conversions can occur in some operations.

```
# Python: raises TypeError – no implicit conversion "hello" + 3 # TypeError: can only  
concatenate str (not 'int') to str # Fix: explicit conversion "hello" + str(3) # 'hello3'
```

9. FastAPI — Beginner

Q: Q56. What is FastAPI?

Modern Python web framework based on ASGI, Starlette (web), and Pydantic (validation). Features: automatic validation, OpenAPI docs, async support, type-driven design.

Q: Q57. FastAPI vs Flask?

- **Flask** — Minimal, manual validation/docs, WSGI by default
- **FastAPI** — Built-in validation, auto docs, async-friendly, type-hint API design

Q: Q58. What is ASGI?

Asynchronous Server Gateway Interface — modern Python interface supporting async requests, websockets, long-lived connections. Better concurrency than WSGI for I/O-heavy apps.

Q: Q59. Role of Pydantic in FastAPI?

Request validation, response serialization, JSON→Python parsing, field type enforcement.

```
from pydantic import BaseModel class UserCreate(BaseModel): name: str email: str
```

Q: Q60. Path params vs Query params vs Request body?

```
@app.get('/users/{user_id}') # path param def get_user(user_id: int): ...  
@app.get('/products') # query params def list(page: int = 1): ... @app.post('/users') #  
request body def create(user: UserCreate): ...
```

Q: Q61. What is response_model?

Defines the response schema — hides internal fields, ensures shape consistency, improves docs.

```
@app.get('/users/{id}', response_model=UserResponse) def get_user(id: int): ...
```

Q: Q62. What is APIRouter?

Splits endpoints into modules. Use prefix and tags to organize, then include in main app.

```
router = APIRouter(prefix='/users', tags=['users']) @router.get('/') def list_users():  
return []
```

10. FastAPI — Intermediate

Q: Q63. What is Depends() / Dependency Injection?

Provides reusable dependencies (DB session, auth user, config) to routes without manual wiring.

```
def get_db(): db = SessionLocal() try: yield db finally: db.close() @app.get('/users')
def list_users(db = Depends(get_db)): ...
```

Q: Q64. def vs async def in FastAPI?

- Use **async def** when underlying libraries are async (async DB drivers, async HTTP clients)
- Use **def** when stack is synchronous

Note: Calling blocking code inside async def blocks the event loop — defeats the purpose.

Q: Q65. What is middleware?

Code that runs around every request/response. Use for: logging, timing, correlation IDs, CORS, auth.

```
@app.middleware('http') async def log_requests(request, call_next): response = await
call_next(request) return response
```

Q: Q66. What are BackgroundTasks?

Run lightweight tasks after returning the response (send email, audit log, cache refresh).

Note: Not for heavy/long-running work — use Celery/RQ/workers instead.

```
@app.post('/orders') def create_order(background_tasks: BackgroundTasks):
background_tasks.add_task(send_email, 'user@example.com') return {'message': 'created'}
```

Q: Q67. What is HTTPException?

```
from fastapi import HTTPException if not user: raise HTTPException(status_code=404,
detail='User not found')
```

Q: Q68. Large FastAPI project structure?

```
app/ main.py api/routes/ users.py | orders.py schemas/ user.py | order.py models/
services/ user_service.py repositories/ db/session.py core/ config.py | security.py |
logging.py
```

11. FastAPI — Advanced

Q: Q69. JWT Authentication in FastAPI?

- Login: verify credentials → create JWT → return token
- Protected route: extract bearer token via OAuth2PasswordBearer
- Validate signature/expiry → decode claims → fetch user → authorize
- Libraries: python-jose for JWT, passlib/bcrypt for password hashing

Q: Q70. Long-running tasks in FastAPI?

Enqueue to Celery/worker queue, return job ID immediately, client polls or uses websocket.

Use cases: document ingestion, embeddings generation, AI batch processing

Q: Q71. Streaming responses in FastAPI?

```
from fastapi.responses import StreamingResponse def generate(): for i in range(5): yield f'chunk {i}\n' @app.get('/stream') def stream(): return StreamingResponse(generate(), media_type='text/plain')
```

For LLM apps: stream tokens/chunks as they arrive from the model.

Q: Q56. Redis caching in FastAPI?

- Create Redis client at app startup
- Cache expensive responses with keys: product:123, search:q=phone&page;=1
- Set TTL, invalidate on update
- Use for: catalog, expensive search results, rate limiting counters

Q: Q57. Scaling FastAPI in production?

- Horizontal scaling — multiple instances behind load balancer
- Stateless app — store session/state in Redis/DB
- Connection pooling + async where appropriate
- Background workers for heavy tasks
- gunicorn + uvicorn.workers.UvicornWorker → containers → ECS/Kubernetes

Q: Q58. Testing FastAPI endpoints?

```
from fastapi.testclient import TestClient from app.main import app client = TestClient(app) def test_health(): response = client.get('/health') assert response.status_code == 200
```

Use dependency overrides to avoid hitting real DB/auth in tests.

Q: Q59. How do you handle long-running AI tasks in FastAPI?

Use a job/worker pattern — never keep the HTTP request open for long-running AI work.

- Request comes in → create job row with status

- Enqueue task to Celery / RQ / worker queue
- Return job ID immediately to client
- Worker processes: ingestion / embeddings / AI batch work
- Client polls GET /jobs/{id}/status or receives websocket update

Q: Q60. How do you stream LLM responses in FastAPI?

```
from fastapi.responses import StreamingResponse
async def token_generator(prompt: str):
    async for chunk in llm_client.stream(prompt):
        yield chunk.content
@app.post('/chat/stream')
async def stream_chat(request: ChatRequest):
    return StreamingResponse(
        token_generator(request.message),
        media_type='text/event-stream'
    )
```

Also works with server-sent events (SSE) or websockets for bi-directional streaming.

Q: Q61. Production FastAPI AI project structure?

```
app/
  api/
    routes/
      chat.py | documents.py | search.py
    services/
      repositories/
        models/
          schemas/
            llm/
              client.py # LLM provider abstraction
      streaming.py
      prompts/
        system_prompts.py
      templates/
        rag/
          retriever.py
          reranker.py
          chunker.py
    workers/
      ingestion.py # background ingestion jobs
      embedding.py
    core/
      config.py | logging.py |
      tracing.py
  db/
  tests/
```

Key idea: routes stay thin, services orchestrate, llm/ and rag/ are isolated layers, workers handle async heavy work.

12. Senior Scenario Questions

Scenario 1 — Design a product/order API

- Routes layer for HTTP endpoints
- Pydantic schemas for request/response
- Service layer for business rules
- Repository layer for DB access
- DB dependency per request
- JWT auth for protected actions
- Pagination / filtering / sorting
- Transaction boundaries for order creation
- Idempotency for payment/order endpoints
- Logging + monitoring

Scenario 2 — Build a chatbot backend in FastAPI

- Endpoints: POST /chat, GET /conversation/{id}, GET /stream-chat
- Route validates → service orchestrates → LLM client abstraction
- PostgreSQL for conversation metadata
- Redis for session/context
- Vector DB / pgvector for retrieval
- Token streaming, history truncation, rate limiting, tracing

Scenario 3 — Document upload + ingestion API

- POST /documents/upload → store in object storage
- Enqueue ingestion job to background worker
- Worker: extract text → chunk → embed → index
- States: uploaded → queued → processing → indexed → failed
- GET /documents/{id}/status for polling

Scenario 4 — Recommendation API

- POST /recommend with user_id + query
- Fetch user context/profile
- Candidate retrieval from search/catalog
- Ranking + business rules
- Cache repeated results
- Log latency and ranking decisions

13. Coding Questions

1. Count word frequency

```
def word_count(text: str): counts = {} for word in text.split(): counts[word] = counts.get(word, 0) + 1 return counts
```

2. Remove duplicates preserving order

```
def dedupe(items): seen = set() result = [] for item in items: if item not in seen: seen.add(item) result.append(item) return result
```

3. Decorator to measure execution time

```
import time from functools import wraps def timeit(func): @wraps(func) def wrapper(*args, **kwargs): start = time.time() result = func(*args, **kwargs) print(f'{{func.__name__}} took {{time.time() - start:.4f}}s') return result return wrapper
```

4. Flatten nested list

```
def flatten(lst): result = [] for item in lst: if isinstance(item, list): result.extend(flatten(item)) else: result.append(item) return result
```

5. POST endpoint with Pydantic validation

```
from fastapi import FastAPI from pydantic import BaseModel, EmailStr app = FastAPI() class UserCreate(BaseModel): name: str email: EmailStr @app.post('/users') def create_user(user: UserCreate): return {'message': 'user created', 'user': user}
```

6. DB dependency pattern

```
from sqlalchemy.orm import Session from fastapi import Depends def get_db(): db = SessionLocal() try: yield db finally: db.close() @app.get('/users') def get_users(db: Session = Depends(get_db)): return []
```

14. What Senior Interviewers Expect

For 8 years experience, interviewers expect tradeoffs and production concerns — not just definitions.

Async def in FastAPI

- **Weak:** Async is faster.
- **Strong:** async def is useful when the underlying work is I/O-bound and libraries are async-aware. It allows the event loop to switch to other requests while waiting. But if the route calls blocking synchronous code, using async def alone won't help and can actually block the event loop. I choose async based on the entire request path, not just the route signature.

FastAPI project structure

- **Weak:** I make routers and models.
- **Strong:** I keep the HTTP layer thin and move business logic into services. Repositories handle persistence, schemas define request/response contracts, and dependencies inject DB sessions and auth context. This keeps endpoints testable, prevents route handlers becoming business-logic heavy, and makes logic reusable across background jobs, CLI tasks, and future services.

Long-running tasks

- **Weak:** I use background tasks.
- **Strong:** If it's lightweight post-response work like sending an email, FastAPI BackgroundTasks is enough. But for ingestion, AI processing, or retry-critical workflows, I use a proper queue/worker model because I need durability, retry semantics, observability, and to avoid tying heavy work to web workers.

Must-Know Python Checklist

- ■ Mutable vs immutable
- ■ == vs is
- ■ Decorators
- ■ Generators / yield
- ■ Shallow vs deep copy
- ■ classmethod vs staticmethod
- ■ Exception handling
- ■ async/await + event loop
- ■ GIL
- ■ Context managers
- ■ List comprehension
- ■ LEGB scope rule
- ■ Memory management & GC

Must-Know FastAPI Checklist

- ■ FastAPI vs Flask
- ■ Pydantic request/response models
- ■ Path/query/body params
- ■ Depends() and dependency injection
- ■ def vs async def
- ■ DB session management
- ■ Middleware / background tasks
- ■ Exception handling
- ■ JWT auth
- ■ Project structure / service layer / repositories
- ■ Long-running AI tasks (job/worker pattern)
- ■ Streaming LLM responses (StreamingResponse / SSE)
- ■ Production AI project structure (llm/, rag/, workers/, prompts/)